arXiv:2206.07389v1 [cs.CV] 15 Jun 2022

Ultra Fast Deep Lane Detection with Hybrid Anchor Driven Ordinal Classification

Zequn Qin, Pengyi Zhang, and Xi Li, *Senior Member, IEEE*

Abstract—Modern methods mainly regard lane detection as a problem of pixel-wise segmentation, which is struggling to address the problems of efficiency and challenging scenarios like severe occlusions and extreme lighting conditions. Inspired by human perception, the recognition of lanes under severe occlusions and extreme lighting conditions is mainly based on contextual and global information. Motivated by this observation, we propose a novel, simple, yet effective formulation aiming at ultra fast speed and the problem of challenging scenarios. Specifically, we treat the process of lane detection as an anchor-driven ordinal classification problem using global features. First, we represent lanes with sparse coordinates on a series of hybrid (row and column) anchors. With the help of the anchor-driven representation, we then reformulate the lane detection task as an ordinal classification problem to get the coordinates of lanes. Our method could significantly reduce the computational cost with the anchor-driven representation. Using the large receptive field property of the ordinal classification formulation, we could also handle challenging scenarios. Extensive experiments on four lane detection datasets show that our method could achieve state-of-the-art performance in terms of both speed and accuracy. A lightweight version could even achieve 300+ frames per second(FPS). Our code is at https://github.com/cfzd/Ultra-Fast-Lane-Detection-v2.

Index Terms—Lane detection, hybrid anchor representation, anchor-driven ordinal classification.

1 Introduction

Lane detection is a fundamental component in autonomous driving and advanced driver-assistance systems (ADAS) [1], which distinguishes and locates the lane markings on the road. Although great success has been achieved with the deep learning models, there are still some important and challenging problems to be addressed.

The first one is the efficiency problem. In practice, the lane detection algorithm is heavily executed to provide instant perception results with constrained vehicle computing devices for the downstream tasks, which require fast detection speed. Moreover, previous lane detection methods [2], [3], [4], [5] are mainly based on segmentation, which is formulated in a dense bottom-up learning pipeline making it hard to achieve fast speed.

Besides the efficiency problem, another challenge is the no-visual-clue problem, as shown in Fig. 1. The lane detection task [2], [6] is to find the location of lanes, no matter whether the lanes are visible or not. In this way, how to handle the scenarios with severe occlusions and extreme lighting conditions that have no visible information is a major difficulty in the lane detection task. To alleviate this problem, additional clues that could potentially hint at the detection are crucial. For example, road shape, cars heading direction trends, non-occluded endpoints of lanes, etc., could benefit the detection. In order to make it possible to use the additional clues, enlarging the receptive field to utilize more information is preferable to lane detection.

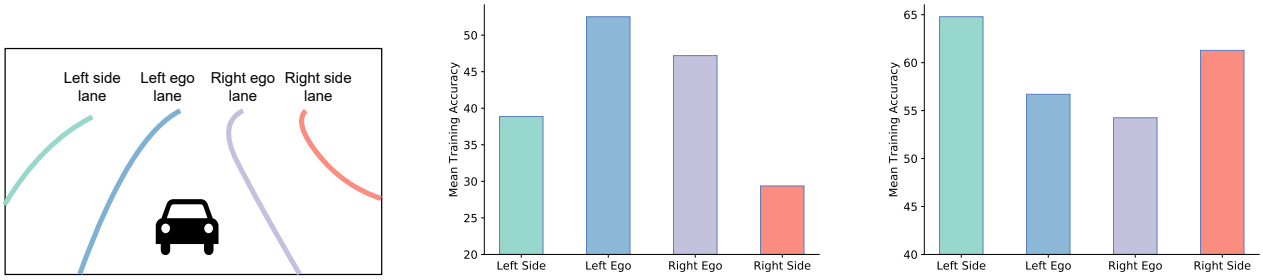
Fig. 1. Illustration of difficulties in lane detection. Different lanes are marked with different colors. Most challenging scenarios are severely occluded or distorted with various lighting conditions, resulting in little or no visual clues of lanes that can be used for lane detection.

This raises a natural question: can we find a fast and global formulation having large receptive field for the lane detection task? With the above motivations, we propose a sparse top-down formulation to address the efficiency and no-visual-clue problems. First, we propose a novel row-anchor-driven representation for lanes. A lane can be represented by the coordinates on a series of predefined row anchors, as shown in Fig. 3. Since a lane can be well represented by a small set of key points (in a fixed sparse row anchor system), the efficiency problem can be addressed with the sparsity of the anchor-driven representation. Second, we propose to use a classification-based manner to learn the coordinates of lanes with the anchor-driven representation. Using the classification-based manner (working with the whole global feature), the receptive field is as large as the whole input. It enables the network to better capture the global and long-range information for lane detection, and the problem of no-visual-clue can be relieved effectively.

Furthermore, we extend the lane representation with

- *Z. Qin and P. Zhang are with the College of Computer Science and Technology, Zhejiang University, Hangzhou, Zhejiang, China, 310007.*
- *X. Li is with the College of Computer Science and Technology, Zhejiang University, Hangzhou, China, 310007; Shanghai Institute for Advanced Study, Zhejiang University, Shanghai, China, 201203; and the Zhejiang–Singapore Innovation and AI Joint Research Lab.*
E-mail: zequnqin@gmail.com, pyzhang@zju.edu.cn, xilizju@zju.edu.cn

(Corresponding author: Xi Li)



(a) Lane definition (b) Lane-wise Acc. with row anchor (c) Lane-wise Acc. with column anchor

Fig. 2. Illustration of the localization accuracy for each kind of lane. (a) shows the definition of lanes by the CULane dataset [2]. (b) is the lane-wise accuracy with the row anchor system. (c) illustrates the lane-wise accuracy with the column anchor system. We can see that for ego lanes, the row anchor system gains better performance, while the column anchor system gains better performance for side lanes.

row anchor to a hybrid anchor system in this work. Our observation is that the row anchor systems may not work well for all kinds of lanes, and it could cause a magnified localization problem. As shown in Figs. 2a and 2b, the localization accuracies for side lanes are significantly lower than those for the ego lanes when using row anchors. What if we use column anchors? In Fig. 2c, we can see an opposite phenomenon that the column anchor system has worse localization ability for ego lanes. We name this phenomenon *magnified localization error problem*. This problem makes row anchors difficult to localize the horizontal (side lanes) lanes and similarly makes column anchors hard to localize the vertical lanes (ego lanes). With the above observation, we propose to use hybrid (row and column) anchors to represent different lanes separately. Specifically, we use row anchor for ego lanes and column anchor for side lanes. In this way, the magnified localization error problem can be relieved, and the performance can be improved.

With the hybrid anchor system, a lane can be represented by the coordinates on the anchor system. How to effectively learn these coordinates is another important problem. The most straightforward method is to use regression. Usually, the regression methods work for a local range prediction setting [7], [8], [9], [10], [11], [12], [13], [14] and are relatively weak in modeling long-range and global localization [15], [16], [17]. To cope with global range prediction, we propose to learn the coordinates of lanes in a classification-based manner, which represents different coordinates using different classes. In this work, we further extend the original classification to the ordinal classification. In the ordinal classification, adjacent classes have close and ordinal relationships, which is different from the original classification. For example, in the ImageNet [18] classification task, the 7th class is stingray (a kind of fish) while the 8th class is cock. In our work, classes are ordered (e.g., the lane coordinates for the 8th class are always spatially on the right of those for the 7th class). Another property of ordinal classification is that the space of classes are continuous. For example, non-integer class like 7.5th class is meaningful, and it can be regarded as an intermediate class between 7th and 8th classes. To realize ordinal classification, we propose two loss functions together to modeling the ordinal relationship between classes, including a base classification loss and a mathematical expectation loss. Utilizing the ordinal relationship and continuous class space properties, we can use the mathematical expectation instead of argmax to get the continuous class [19] of prediction. The expectation loss is to constrain the predicted continuous class to equal the ground truth. By simultaneously constraining the base and expectation losses, the output could have a better ordinal relationship and benefit the localization of lanes.

In summary, the main contributions of our work are threefold. 1) We propose a novel, simple, yet effective formulation for lane detection. Compared with the previous methods, our method represents lanes as anchor-based coordinates, and the coordinates are learned in a classification-based manner. This formulation is ultra-fast and effective in solving the no-visual-clue problem. 2) Based on the proposed formulation, we propose a hybrid anchor system, which further extends the previous row anchor system and could effectively reduce the localization error. Moreover, classification-based learning is further extended to an ordinal classification problem, which utilizes the natural ordinal relationship in classification-based localization. 3) The proposed method achieves state-of-the-art speed and performance. Our fastest model could achieve 300+ FPS with comparable performance to the state-of-the-art.

This paper is an extension of our previous conference publication [20]. Compared with the conference version, this paper has extensions as follows:

- *Hybrid Anchor System* With the observation of the magnified error problem, we propose a new hybrid anchor system that could effectively reduce localization error compared with the previous publication.
- *Ordinal Classification Losses* We propose the new loss functions that treat lane localization as an ordinal classification problem, which further improves the performance.
- *Presentation & Experiments* Most of the paper is rewritten to give clearer presentations and illustrations. We provide more analyses, visualizations, and results to better cover the space of our work. Stronger results with 6.3 points of performance improvement at the same speed are also provided in this version.

2 Related Work

2.1 Lane Detection with Bottom-up Modeling

Traditional approaches usually tackle the lane detection problem using low-level image processing techniques. By

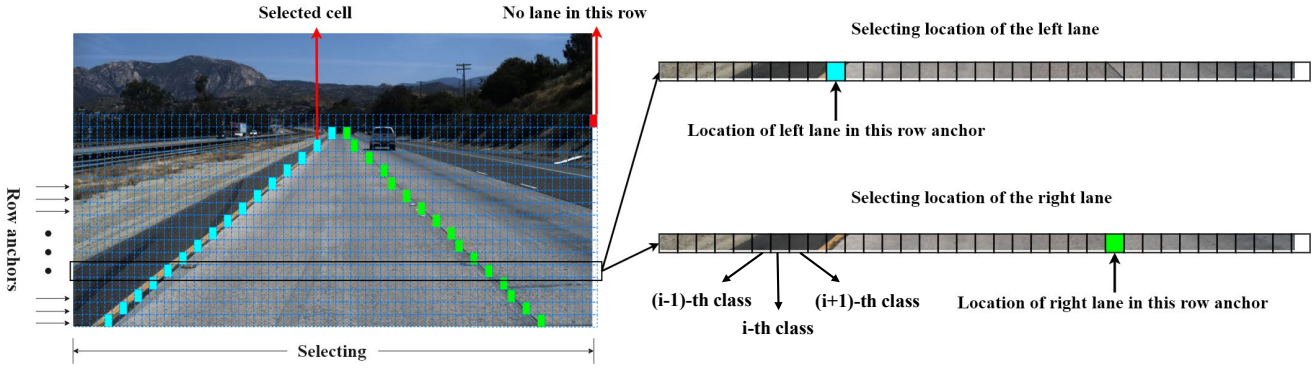


Fig. 3. Illustration of the row anchor system. Row anchors are the predefined sparse row locations, and lanes can be represented as coordinates (in a classification-based manner) on the row anchors.

using low-level image processing, traditional methods work in a bottom-up manner inherently. Their main idea is to take advantage of visual clues through image processing like the HSI color model [21] and edge extraction algorithms [22], [23]. Gold [24] is one of the earliest attempt that detects lanes and obstacles using edge extraction algorithms from a stereo vision system. Besides using the features from different color models and edge extraction methods, [25] propose to use projective geometry and inverse perspective mapping to utilize the prior information that lanes are typically parallel in the real world. Although many methods try different traditional features for lanes, the semantic information from the low-level image processing is still relatively insufficient in the complex scenarios. In this way, tracking is another popular post-processing solution [26], [27] to enhance the robustness. Besides tracking, markov and conditional random fields [28] are also used as post-processing methods. Also, some methods [29], [30], [31] adopting learning mechanisms (like template matching, decision tree, and support vector machine) are proposed.

With the development of deep learning, some methods [32], [33] based on deep neural networks show the superiority in lane detection. These methods usually tackle the lane detection task using heat maps that indicate the existence and locations of lanes. Following these early attempts, the mainstream methods start to treat lane detection as a segmentation problem. For instance, VPGNet [4] proposes a multi-task segmentation network guided by vanishing points for lanes and road markings detection. To enlarge the receptive field of pixel-wise segmentation and boost the performance, SCNN [2] utilizes a special convolution operation in the segmentation module. It aggregates information from different dimensions via processing sliced features and adding them together one by one, which is similar to recurrent neural networks. RONELD [34] proposes an enhancement method for SCNN by finding and constructing straight and curved active lanes separately. RESA [35] also proposes a similar approach by recurrent feature-shift to enlarge the receptive field. Due to the heavy computational load of segmentation methods, some works try to explore lightweight methods for real-time applications. Self-attention distillation (SAD) [3] applies an attention distillation mechanism, in which high and low layers' attentions are respectively treated as teachers and students. IntRA-KD [36] also uses inter-region affinity distillation to boost the performance of the student network. In this way, a shallow network could have a similar performance as a deep network by attention distillation. CurveLane-NAS [6] introduces neural architecture search techniques to search segmentation networks tailored for lane detection. In LaneAF [37], this work proposes to detect lanes by voting in affinity fields based on the segmentation. FOLOLane [38] proposes to model local patterns and achieves global prediction of global structure with a global geometry decoder in a bottom-up manner.

2.2 Lane Detection with Top-down Modeling

Besides the mainstream segmentation formulation, some works also try to explore other formulations for lane detection. In [39], a long short-term memory (LSTM) network is adopted to deal with the long line structure of lanes. With the same principle, Fast-Draw [40] predicts the direction of lanes at each lane point, then draws them out sequentially. In [41], the problem of lane detection is regarded as instance segmentation by clustering binary segments. E2E [42] proposes to detect lanes by differentiable least-squares fitting and directly predicts the polynomial coefficients of lanes. Similarly, Polylanenet [43] and LSTR [44] also propose to predict the lanes' polynomial coefficients by deep polynomial regression and Transformer [45] respectively. LaneATT [46] proposes to use object detection pipelines that regard lines in the image as anchors. Then it classifies and localizes lanes from the dense line anchors. Following the idea of utilizing vanishing points prior [4] and object detection pipelines [46], [47] proposes to use line anchors guided by vanishing points. Different from the 2D view of previous works, there are also many methods [48], [49], [50], [51] trying to detect lanes in 3D.

Different from the previous bottom-up works, our method is a top-down modeling method. By top-down modeling, the method could naturally focus more on the global information, which is beneficial to the no-visual-clue problem. Compared with the previous top-down methods, our method is aiming at a new lane detection formulation with the row and hybrid anchor lane representations, which could greatly reduce the learning difficulty and speed-up detection. With the proposed formulation in the previous conference version [20], our works have been successfully adopted and generalized to other methods [52].

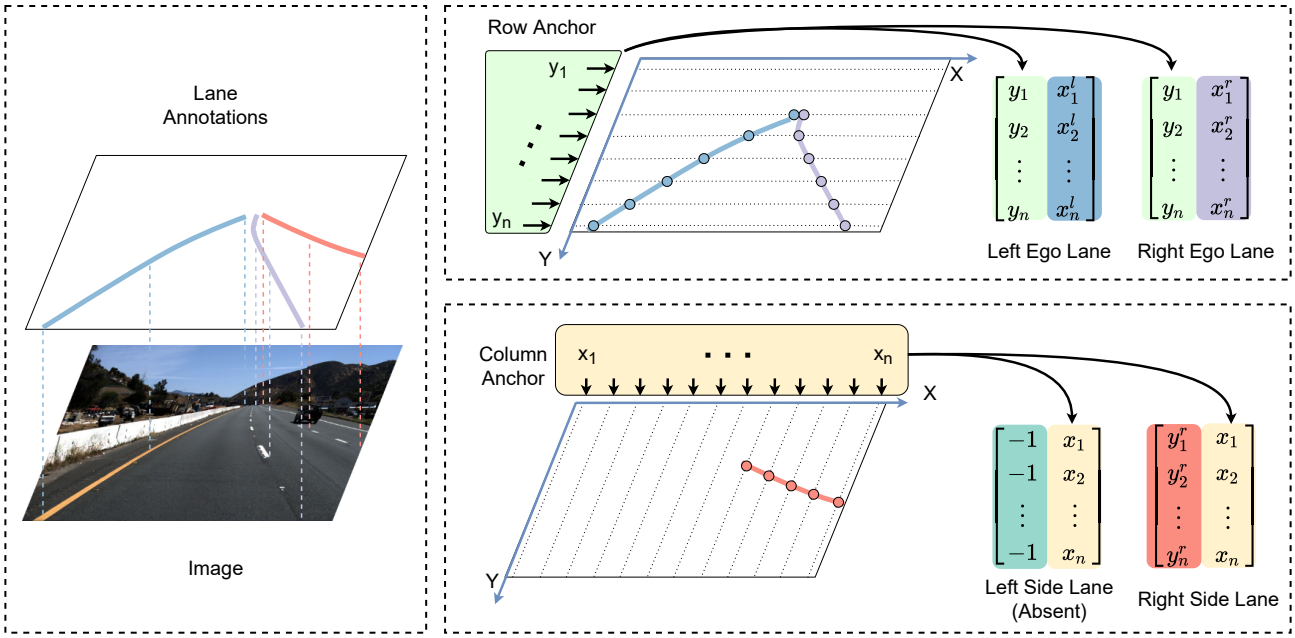


Fig. 4. Lane representation with the hybrid anchor system. The input image and its annotations are shown on the left. For the ego lanes, we use row anchors to represent the location of lane markings. For side lanes, the column anchors are adopted. With the hybrid anchor system, the lanes can be simply represented as four matrices corresponding to the four lanes as shown on the right. Moreover, the fixed and shared coordinates of anchors can be left out. For the absent locations, we use -1 to indicate them.

3 Ultra Fast Lane Detection

In this section, we describe the details of our method. First, we demonstrate the representation of lanes in coordinates on the proposed hybrid anchor system. Second, we show the design of deep network architecture and corresponding ordinal classification losses. Last, the complexity analysis is elaborated.

3.1 Lane Representation with Anchors

In order to represent lanes, we introduce the row anchors for lane detection, as shown in Fig. 3. Lanes are represented with points on the row anchors. However, the row anchor system might cause a magnified localization error problem, as shown in Fig. 2. In this way, we further extend the row anchor system to a hybrid anchor system.

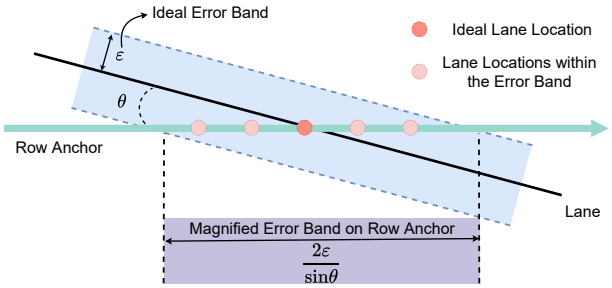


Fig. 5. Illustration of the magnified localization error problem. The blue area represents the ideal localization error band (the minimal inevitable error that a network could have. It is possibly caused by the bias of a network, the error introduced by the annotator, etc.) without an anchor system. The purple area shows the magnified error band introduced by the anchor system. We can see that when the angles between lanes and anchors are small, the localization error can be magnified tremendously. The magnified localization error problem also holds for column anchors.

The reason for this problem is shown in Fig. 5. Suppose the ideal minimal localization error without any anchor system is ε, which can be caused by network bias, annotation error, etc. We can see that the error band on the row anchor system is multiplied by a factor of $\frac{1}{\sin\theta}$. When the angle θ between the lane and anchor is small, the magnifying factor $\frac{1}{\sin\theta}$ would go to infinity. For example, when a lane is strictly horizontal, it is impossible to represent the lane with the row anchor system. This problem makes the row anchors hard to localize the lanes which are more horizontal (typically side lanes), and similarly, it makes the column anchors hard to localize the lanes which are more vertical (typically ego lanes). Instead, when the lanes and anchors are vertical, the error introduced by the anchor system is minimal ($\theta = 0$ in this case), and it equals the ideal localization error ε.

Motivated by the above observations, we further propose to use hybrid anchors to represent lanes. For different kinds of lanes, we use different anchor systems to reduce the magnified localization error. Specifically, the rule is that: a lane can only be assigned with one kind of anchor, and the more vertical anchor type to the lane is chosen. In practice, lane detection datasets like CULane [2] and TuSimple [53] only annotate the two ego lanes and the two side lanes, as shown in Fig. 2a. In this way, we use row anchors for ego lanes and column anchors for side lanes, and the magnified localization error problem can be relieved by the hybrid anchor system.

With the hybrid anchor system, we can represent lanes as a series of coordinates on the anchor, as shown in Fig. 4. Denote N_{row} as the number of row anchors and N_{col} as the number of column anchors. For each lane, we first assign the corresponding anchor system, which has the minimal localization error. Then we calculate the line–line intersection between the lane and each anchor, and record

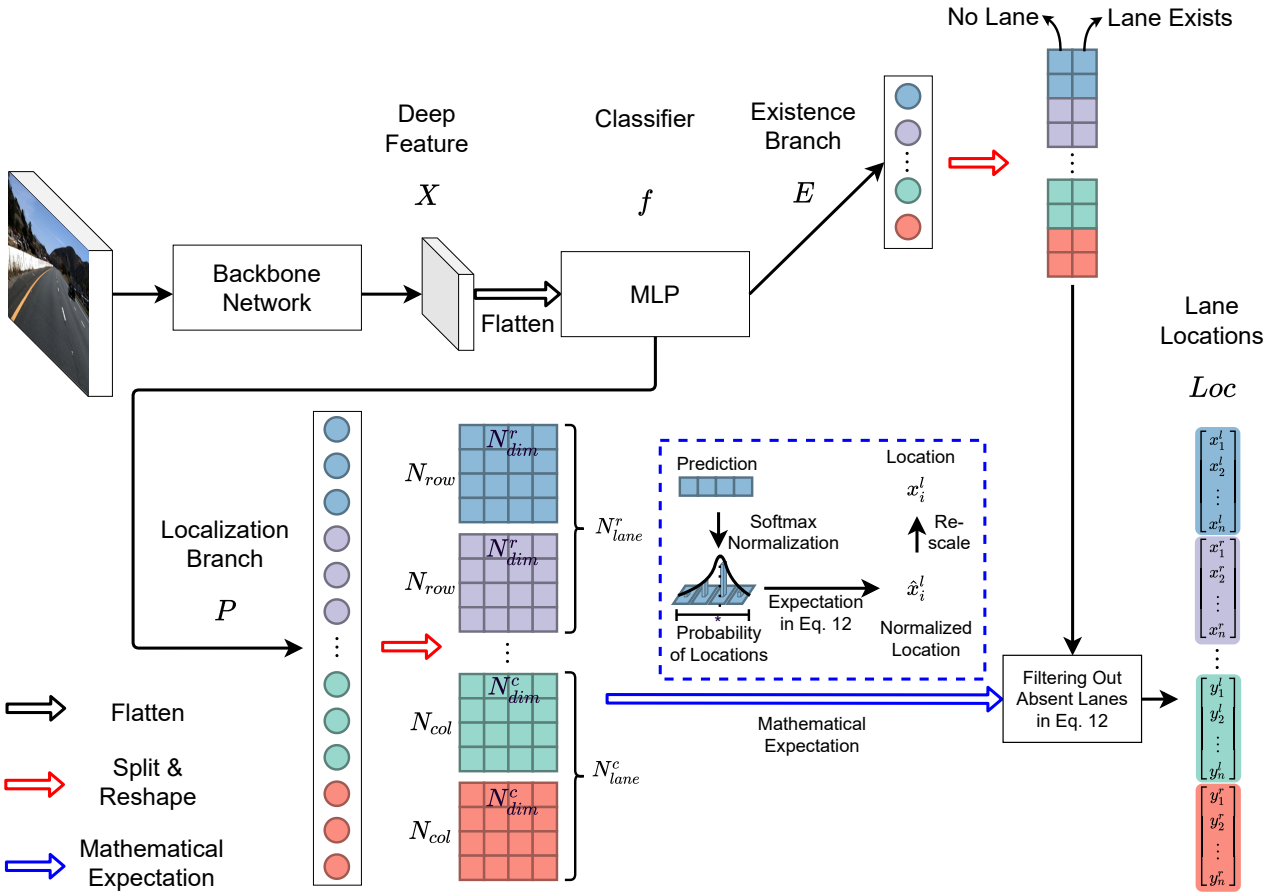


Fig. 6. Illustration of the network architecture. The input image is first sent to a backbone network to get the deep feature. Then the deep feature is flattened and fed into a classifier, which has two output branches. The first localization branch is to learn the coordinates on the hybrid anchors with classification-based representation. The second existence branch is to predict the existence of each coordinate on the hybrid anchors. After obtaining the localization output, we use expectation instead of argmax to get the coordinates of lanes.

the coordinate of the intersection point. If the lane has no intersection between certain anchors, the coordinates will be set to -1. Suppose the number of lanes assigned to row anchors is N^r_{lane} and the one for column anchors is N^c_{lane}. The lanes in an image can be represented by a fixed-size target T where every element is either the coordinate of lane or -1, and its length is $N_{row} \times N^r_{lane} + N_{col} \times N^c_{lane}$. T can be divided into two parts T^r and T^c, which correspond to the parts on row and column anchors, and the sizes are $N_{row} \times N^r_{lane}$ and $N_{col} \times N^c_{lane}$ respectively.

3.2 Anchor-driven Network Design

With the help of lane representation with hybrid anchor, our goal of designing networks is to learn the fixed-size targets T^r and T^c with classification. To learn T^r and T^c with classification, we map different coordinates in T^r and T^c to distinct classes. Suppose T^r and T^c are normalized (the elements of T^r and T^c range from 0 to 1 or equal -1, i.e., the "no lane" case), and the numbers of classes are N^r_{dim} and N^c_{dim}. The mapping can be written as:

$$\begin{cases} T^r_{cls_i,j} = \left\lfloor T^r_{i,j} N^r_{dim} \right\rfloor, \\ T^c_{cls_m,n} = \left\lfloor T^c_{m,n} N^c_{dim} \right\rfloor, \end{cases} \quad (1)$$
$$s.t. \quad i \in \{1, \cdots, N_{row}\}, j \in \{1, \cdots, N^r_{lane}\},$$
$$m \in \{1, \cdots, N_{col}\}, n \in \{1, \cdots, N^c_{lane}\},$$

TABLE 1
Notations

Variable	Type	Definition
H	Scalar	Height of image
W	Scalar	Width of image
N_{row}	Scalar	Number of row anchors
N_{col}	Scalar	Number of col anchors
N^r_{dim}	Scalar	Classification dimension on row anchor
N^c_{dim}	Scalar	Classification dimension on col anchor
N^r_{lane}	Scalar	Number of lanes on row anchor
N^c_{lane}	Scalar	Number of lanes on col anchor
f	Function	The classifier
X	Tensor	The global feature of image
P	Tensor	Prediction of localization branch
E	Tensor	Prediction of existence branch
T	Tensor	Lane representation with hybrid anchor
T_{cls}	Tensor	Learning target of localization branch
T_{ext}	Tensor	Learning target of existence branch
$Prob$	Tensor	Probability of each location
Loc	Tensor	Location of lanes

For variables P, E, T, T_{cls}, T_{ext}, $Prob$, and Loc, a superscript $\cdot^r$, like P^r, represents the parts corresponding the row anchor, while $\cdot^c$ corresponds the column anchor.

in which T^r_{cls} and T^c_{cls} are the mapped class labels of the coordinates, $\lfloor \cdot \rfloor$ is the floor operation, and $T^r_{cls_i,j}$ is the

element in the i-th row, j-th column of T^r_{cls}. In this way, we could convert the learning of coordinate on the hybrid anchor to two classification problems with dimensions of N^r_{dim} and N^c_{dim}, respectively. For the no lane case, i.e., $T^r_{i,j}$ or $T^c_{m,n}$ equals -1, we use an additional two-way classification to indicate:

$$T^r_{ext_i,j} = \begin{cases} 1, & \text{if } T^r_{i,j} \neq -1 \\ 0, & \text{otherwise} \end{cases}, \quad s.t. \quad i \in \{1, \cdots, N_{row}\}, j \in \{1, \cdots, N^r_{lane}\}, \tag{2}$$

in which T^r_{ext} is the class label of the coordinates' existence, and $T^r_{ext_i,j}$ is the element in the i-th row, j-th column of T^r_{ext}. The existence targets for column anchor T^c_{ext} is similar:

$$T^c_{ext_m,n} = \begin{cases} 1, & \text{if } T^c_{m,n} \neq -1 \\ 0, & \text{otherwise} \end{cases}, \quad s.t. \quad m \in \{1, \cdots, N_{col}\}, n \in \{1, \cdots, N^c_{lane}\}. \tag{3}$$

With the above derivation, the whole network is to learn the T^r_{cls}, T^c_{cls}, T^r_{ext} and T^c_{ext} with two branches, which are localization and existence branches. Suppose the deep feature of an input image is X, the network can be written as:

$$P, E = f(\text{flatten}(X)), \tag{4}$$

in which P and E are the localization and existence branches, f is the classifier, and flatten$(\cdot)$ is the flatten operation. The outputs of P and E are all composed of two parts (P^r, P^c, E^r and E^c), which correspond to the row and column anchors respectively. The sizes of P^r and P^c are $N^r_{lane} \times N_{row} \times N^r_{dim}$ and $N^c_{lane} \times N_{col} \times N^c_{dim}$ respectively, in which N^r_{dim} and N^c_{dim} are the mapped classification dimensions for row and column anchors. The sizes of E^r and E^c are $N^r_{lane} \times N_{row} \times 2$ and $N^c_{lane} \times N_{col} \times 2$ respectively.

In Eq. 4, we directly flatten the deep features from the backbone and feed them to the classifier. In comparison, conventional classification networks [54], [55], [56], [57] use global average pooling (GAP). The reason why we use flatten instead of GAP is that we find the spatial information is crucial for the classification-based lane detection network. Using GAP would eliminate the spatial information and result in poor performance.

3.3 Ordinal Classification Losses

As we can see in Eq. 1, an essential property is that the classes in the above classification network have an ordinal relationship. In our classification network, adjacent classes are defined to have a close and ordinal relationship, which is different from conventional classification. To better utilize the prior of the ordinal relationship, we propose to use a base classification loss and an expectation loss.

The base classification loss is defined as:

$$L_{cls} = \sum_{i=1}^{N^r_{lane}} \sum_{j=1}^{N_{row}} L_{CE}(P^r_{i,j}, \text{onehot}(T^r_{cls_i,j})) + \sum_{m=1}^{N^c_{lane}} \sum_{n=1}^{N_{col}} L_{CE}(P^c_{m,n}, \text{onehot}(T^c_{cls_m,n})), \tag{5}$$

in which $L_{CE}(\cdot)$ is the cross entropy loss, $P^r_{i,j}$ is the prediction of i-th lane that is assigned to row anchor and j-th row anchor, $T^r_{cls_i,j}$ is the corresponding classification label to $P^r_{i,j}$, $P^c_{m,n}$ is the prediction of m-th lane that is assigned to column anchor and n-th column anchor, $T^c_{cls_m,n}$ is the corresponding classification label to $P^c_{m,n}$, and onehot$(\cdot)$ is the one-hot encoding function.

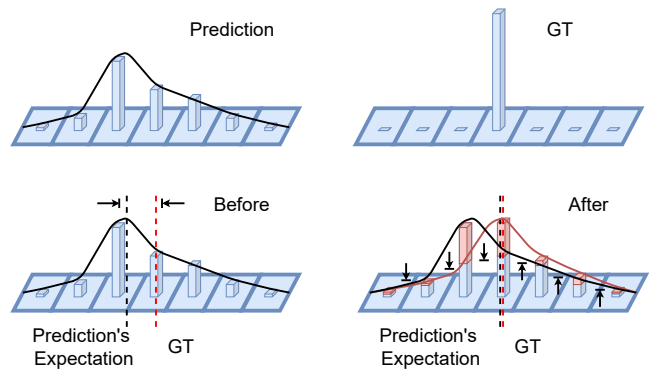


Fig. 7. Illustration of the proposed expectation loss.

Since the classes are ordinal, the expectation of the prediction can be regarded as the mean voting results. For convenience, we denote the expectation as:

$$\begin{cases} Exp^r_{i,j} = \sum_{k=1}^{N^r_{dim}} Prob^r_{i,j}[k] \cdot k, \\ Exp^c_{m,n} = \sum_{l=1}^{N^c_{dim}} Prob^c_{m,n}[l] \cdot l, \end{cases} \tag{6}$$

in which $[\cdot]$ denotes the indexing operator. The $Prob$ is defined as:

$$\begin{cases} Prob^r_{i,j} = \text{softmax}(P^r_{i,j}), \\ Prob^c_{m,n} = \text{softmax}(P^c_{m,n}), \end{cases} \tag{7}$$

In this way, we could constrain the expectation of prediction to get close to the ground truth. Thus, we have the following expectation loss:

$$L_{exp} = \sum_{i=1}^{N^r_{lane}} \sum_{j=1}^{N_{row}} L_1(Exp^r_{i,j}, T^r_{cls_i,j}) + \sum_{m=1}^{N^c_{lane}} \sum_{n=1}^{N_{col}} L_1(Exp^c_{m,n}, T^c_{cls_m,n}), \tag{8}$$

in which $L_1(\cdot)$ is the smooth L1 loss.

The illustration of the expectation loss is shown in Fig. 7. We can see that the expectation loss could push the predicted distribution's mathematical expectation towards the location of ground truth, hence benefits the localization of lanes.

In addition, the loss of existence branch L_{ext} is defined as:

$$L_{ext} = \sum_{i=1}^{N^r_{lane}} \sum_{j=1}^{N_{row}} L_{CE}(E^r_{i,j}, \text{onehot}(T^r_{ext_i,j})) + \sum_{m=1}^{N^c_{lane}} \sum_{n=1}^{N_{col}} L_{CE}(E^c_{m,n}, \text{onehot}(T^c_{ext_m,n})). \tag{9}$$

Finally, the total loss can be written as:

$$L = L_{cls} + \alpha L_{exp} + \beta L_{ext}, \tag{10}$$

in which α and β are the loss coefficients.

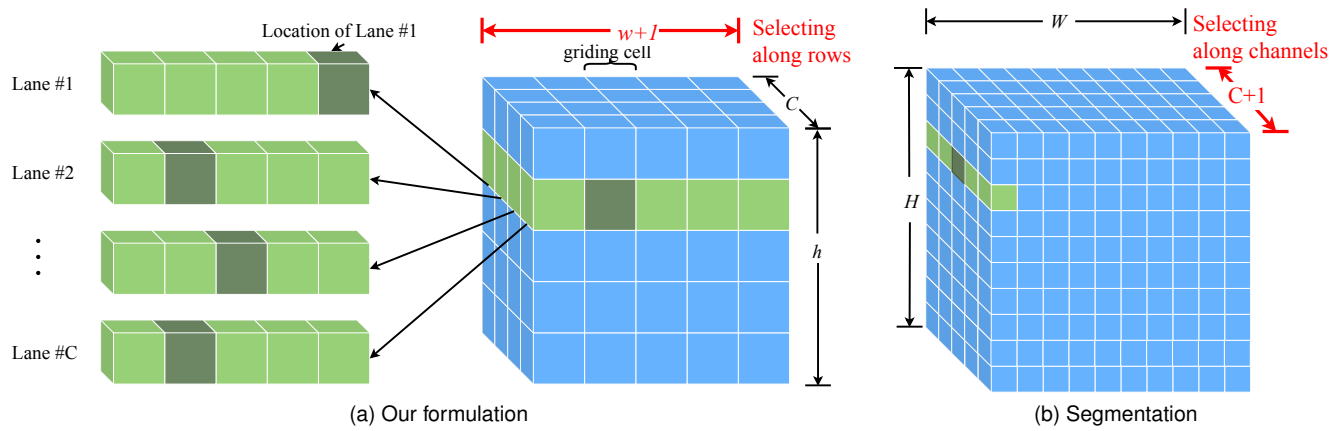


Fig. 8. Illustration of our formulation (row anchor in this case) and conventional segmentation. Taking row anchor as an example; our formulation is selecting locations (grids) on rows, while segmentation is classifying every pixel. The dimensions used for classifying are also different, which is marked with red. The proposed formulation significantly reduces the computational cost. Besides, the proposed formulation uses global feature as input, which has a larger receptive field than segmentation, thus addressing the *no-visual-clue* problem.

3.4 Network Inference

In this section, we show how to get the detection results during inference. Taking row anchor system as an example, suppose $P^r_{i,j}$ and $E^r_{i,j}$ are the predictions of the i-th lane and j-th anchor. Then the length of $P^r_{i,j}$ and $E^r_{i,j}$ are N^r_{dim} and 2 respectively. The probability of each location of lanes can be written as:

$$Prob^r_{i,j} = \text{softmax}(P^r_{i,j}), \tag{11}$$

in which the length of $Prob^r_{i,j}$ is N^r_{dim}. Then the location of lanes is obtained with the mathematical expectation of the predicted distribution. Moreover, predictions of absent lanes will be filtered out according to the prediction of the existence branch:

$$Loc^r_{i,j} = \begin{cases} \sum_{k=1}^{N^r_{dim}} Prob^r_{i,j}[k] \cdot k, & \text{if} \quad E^r_{i,j}[2] > E^r_{i,j}[1] \\ -1, & \text{otherwise} \end{cases}, \tag{12}$$

Finally, the obtained location Loc is scaled to fit the size of the input image. The overall illustration of the network architecture is shown in Fig. 6.

3.5 Analysis and Discussion

In this section, we first analyze the complexity of our method and give the reason why our method could achieve ultra-fast speed.

In order to analyze the complexity, we use segmentation as a baseline. The differences between our formulation (taking row anchor as an example) and segmentation are shown in Fig. 8. It can be seen that our formulation is much simpler than the commonly used segmentation. Suppose the image size is $H \times W$. Since segmentation is pixel-wise classification, it needs to conduct $H \times W$ classifications. For our method, the length of learning target T, which contains the coordinates of lanes on hybrid anchor, is $N_{row} \times N^r_{lane} + N_{col} \times N^c_{lane}$. Since we only need a small number of anchors to represent lanes, and we have $N_{row} \ll H$ and $N_{col} \ll W$. Moreover, N^r_{lane} and N^c_{lane} are the number of lanes assigned to row and column anchors, which is very small compared with other variables. In this way, we have:

$$\underbrace{N_{row} \times N^r_{lane} + N_{col} \times N^c_{lane}}_{\text{\#CLS of our method}} \ll \underbrace{H \times W}_{\text{\#CLS of the segmentation}}, \tag{13}$$

in which #CLS means "the number of classifications". Taking the settings on the CULane [2] dataset as an example, we have $H = 320$, $W = 1600$, $N_{row} = 18$, $N_{col} = 41$, and $N^r_{lane} = N^c_{lane} = 2$. Our method's number of classifications is 118, while the one of segmentation is 5.12×10^5. Taking the classification dimensions into account, the ideal number of calculations of our method is 1.54×10^4, while the one for segmentation is 2.56×10^6. The computational costs of our classification head and segmentation head with ERFNet [36], [58] are 0.04 GMac and 30.86 GMac, respectively.

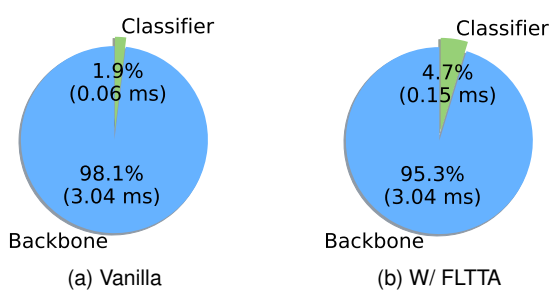


Fig. 9. Pie chart of the forward pass timing for the proposed network with a ResNet-18 backbone. We can see our method is highly efficient, and less than 5% of computations are from the proposed classifier for lane detection.

Besides the theoretical analysis, the real forward pass timing of our method is shown in Fig. 9a. We can see that the backbone takes up most of the time. In contrast, the proposed classification-based lane detection head is highly efficient and only costs less than 5% of the whole inference time.

TABLE 2
Datasets description

Dataset	#Frame	Train	Validation	Test	Resolution	#Lane	#Scenarios	Environment
TuSimple	6,408	3,268	358	2,782	1280×720	≤5	1	highway
CULane	133,235	88,880	9,675	34,680	1640×590	≤4	9	urban and highway
CurveLanes	150,000	100,000	20,000	30,000	2560×1440	≤10	1	urban and highway
LLAMAS	100,042	58,269	20,844	20,929	1276×717	≤4	1	highway

4 EXPERIMENTS [1]

In this section, we demonstrate the effectiveness of our method with extensive experiments. The following sections mainly focus on three aspects: 1) Experimental settings. 2) Ablation studies of our method. 3) Results on four major lane detection datasets.

4.1 Experimental Setting

4.1.1 Datasets

To evaluate our approach, we conduct experiments on four widely used benchmark datasets: TuSimple [53], CULane [2], CurveLanes [6], and LLAMAS [59] datasets. TuSimple dataset is collected with stable lighting conditions on highways. On the contrary, the CULane dataset consists of nine different scenarios, including normal, crowd, curve, dazzle light, night, no line, shadow, and arrow in the urban area. CurveLanes dataset focus on more curved scenarios, and LLAMAS dataset is collected using maps. The detailed information about the datasets can be seen in Table 2.

4.1.2 Evaluation Metrics

The official evaluation metrics of the datasets are different. For the TuSimple dataset, the evaluation metric is accuracy. The accuracy is calculated by:

$$accuracy = \frac{\sum_{clip} C_{clip}}{\sum_{clip} S_{clip}}, \tag{14}$$

in which C_{clip} is the number of lane points predicted correctly and S_{clip} is the total number of ground truth in each clip. Besides the accuracy, we also use the evaluation metric of CULane for all datasets, in which intersection-over-union (IoU) is computed between ground truth and predictions. Predictions with IoUs larger than 0.5 are considered as true positives. F1-measure is taken as the evaluation metric and formulated as follows:

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}, \tag{15}$$

where $Precision = \frac{TP}{TP+FP}$, $Recall = \frac{TP}{TP+FN}$, FP and FN are the false positive and false negative.

4.1.3 Implementation Details

The hyperparameter settings are shown in Table 3. The corresponding ablation studies of number of anchors and classification dimensions can be seen in Sections 4.2.4 and 4.2.5, respectively.

1. We provide a visualization demo video for our method. It can be found at: https://youtu.be/VkvpoHlaMe0 and https://www.bilibili.com/video/BV1gV411x7KX/

TABLE 3
Hyperparameter Settings on Different Datasets.

Dataset	Tusimple	CULane	CurveLanes	LLAMAS
N_{row}	56	18	72	36
N_{col}	40	40	80	40
N^r_{dim}	100	200	200	200
N^c_{dim}	100	100	100	100
N^r_{lane}	2	2	10	4
N^c_{lane}	2	2	10	4

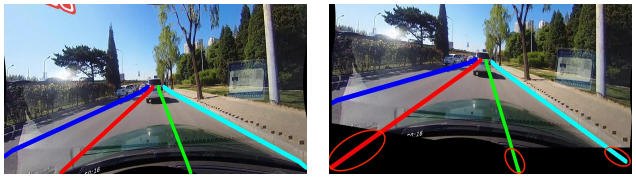
In the optimizing process, images are resized to 1600×320 and 800×320 on CULane and TuSimple, respectively. Loss coefficients α and β in Eq. 10 are set to 0.05 and 1, respectively. The batch size is set to 16 per GPU, and the total number of training epochs is set to 30 for all datasets. The learning rate is set to 0.1 with SGD optimizer and decreases by a factor of 10 at 25th epoch. All models are trained and tested with PyTorch [60] and Nvidia RTX 3090 GPUs.

4.1.4 Feature Level Test Time Augmentation (FLTTA)

In this part, we show the test time augmentation method for our method. Because our method works in a global-feature-based classification way, this offers us an opportunity to conduct fast test time augmentation on the feature level. Different from the test time augmentation (TTA) in the object detection, which usually needs to compute the whole backbone repeatedly during TTA, we conduct FLTTA directly on top of the backbone feature. The backbone feature is computed once, and then it is duplicated with spatial shifts in the up, down, left, and right directions. The five augmented feature copies are then fed into the classifier in a batch. Finally, the outputs are integrated to give better predictions during testing. Since the backbone feature is only computed once and the computation of the classifier works in a batched way that takes advantage of the parallel mechanism of GPUs, the FLTTA works nearly as fast as the vanilla testing without TTA. Fig. 9b shows the forward pass timing pie chart of FLTTA.

4.1.5 Data Augmentation

With simple data augmentations like resize and crop, the proposed method could rapidly overfit the whole training set (nearly 100% training accuracy while poor performance on the testing set). In order to overcome the overfitting problem, we propose a spatial shift data augmentation, which randomly moves the whole image and lane annotations spatially to make the network learn the diverse spatial patterns of lanes. Since some parts of the image and lane annotations are cropped after the spatial shift, we extend the



(a) Original annotation (b) Augmented result

Fig. 10. Demonstration of the spatial shift augmentation. The lane on the right image is extended to maintain the lane structure, which is marked with red ellipse.

lane annotations until the image boundary. Fig. 10 shows the augmentation.

4.2 Ablation Study

In this section, we verify our method with several ablation studies. The experiments are all conducted with the same settings as Section 4.1. All experiments for ablation study are conducted with the backbone of the ResNet-18 network.

4.2.1 Effectiveness of the Hybrid Anchor System

As we have stated in Section 3.1, the row anchor and column anchor system play different roles for lane detection. Motivated by this, we propose a hybrid anchor system, which assigns different lanes to the corresponding anchor type.

To verify the effectiveness of the hybrid anchor systems, we conduct three experiments on CULane. The results are shown in Table 4.

TABLE 4
Comparison between different anchor systems.

Dataset	Row	Col	Hybrid
CULane	66.09 [2]	57.29	74.70

Results are based on ResNet-18. Row means only using row anchor, Col means only using column anchor, and Hybrid corresponds to the hybrid anchor. F1-measure is used for evaluation.

We can see that the hybrid anchor system brings significant improvement compared with row anchors and column anchors, and it shows the effectiveness of hybrid anchors.

4.2.2 Effectiveness of the Ordinal Classification

Our method formulates lane detection as an ordinal classification problem. One natural question is that how about other methods like regression and conventional classification.

For the regression method, we replace the classifier head in our pipeline with a similar regression head. The training loss is replaced with Smooth L1 loss. For the conventional classification method, we use the same classifier head as the one in our pipeline. The differences between classification and ordinal classification are the losses and post-processing. 1) Classification setting only uses cross-entropy loss while ordinal classification setting uses all three losses as in Eq. 10. 2) Classification setting use argmax as its standard post-processing while ordinal classification settings use expectation. The comparison is shown in Table 5.

2. The performance of Row is slightly lower than the original version, which is caused by the post-processing. Details can be found in the supplementary materials.

TABLE 5
Comparison between different localization methods.

Dataset	Regression	Classification	Ordinal classification
CULane	64.29	67.24	74.70

Regression means using the regression as the localization method. Classification is to use the standard classification pipeline to detect lanes. Ordinal classification is associated with the proposed method in this work.

We can see that the classification with the expectation could gain better performance than the standard classification method. Meanwhile, classification-based methods could consistently outperform the regression-based methods.

4.2.3 Ablation of the Ordinal Classification Losses

As described in Section 3.3, we model the lane detection problem as an ordinal classification problem. To verify the effectiveness of the proposed modules, we show the ablation study of the ordinal classification losses.

As shown in Table 6, our proposed constraints of expectation loss improves the performance for lane detection effectively. The proposed expectation loss has a different geometric property compared with the standard cross entropy loss, that the expectation loss is like gradually pushing the prediction's expectation towards the ground truth by reducing the values of logits that are far away from the ground truth and increasing the values of logits that are near the ground truth. Meanwhile, the mean average localization error is also reduced compared with the conventional classification.

TABLE 6
Ablation study of ordinal classification losses on CULane benchmark with a ResNet-18 backbone.

Base loss	Exp loss	F1 (%)
✓		74.18
	✓	69.70
✓	✓	74.70

Base loss is the cross-entropy loss. Exp loss is the expectation loss.

4.2.4 Effects of the Classification Dimensions

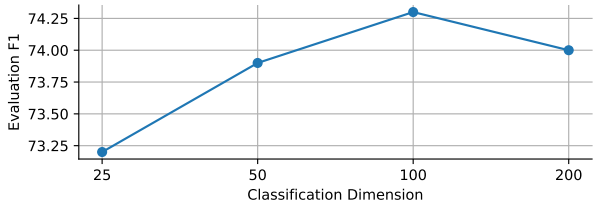
As described in Eq. 4, we use classification-based formulation to detect lanes, and different lane locations are represented with different classes. This raises the question, how many classes are needed to conduct lane detection. To discuss this question, we first set the classification dimension of row anchors N^r_{dim} as 200 and conduct experiments with the classification dimension of column anchors N^c_{dim} as 25, 50, 100, and 200. The results are shown in Fig. 11a. Then, we fix N^c_{dim} as 100, and conduct experiments as N^r_{dim} as 50, 100, 200, and 400. The results are shown in Fig. 11b.

We can see that with the increase of classification dimensions, the performance first increases and then decreases. With a smaller dimension, the classification is inherently easier, but each class represents a wider range of locations,

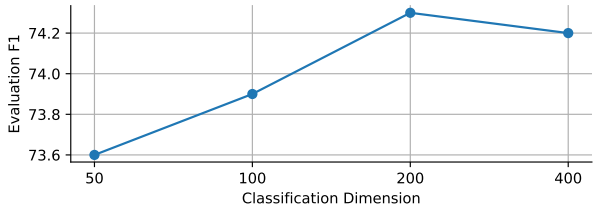
TABLE 7
Comparison of F1-measure and runtime on CULane testing set with IoU threshold=0.5.

Method	Normal	Crowded	Dazzle	Shadow	No line	Arrow	Curve	Crossroad	Night	FPS	Total
SCNN [2]	90.6	69.7	58.5	66.9	43.4	84.1	64.4	1990	66.1	8	71.6
SAD [3]	90.1	68.8	60.2	65.9	41.6	84.0	65.7	1998	66.0	75	70.8
ERFNet-IntRA-KD [36]	-	-	-	-	-	-	-	-	-	100	72.4
SIM-CycleGAN [61]	91.8	71.8	66.4	76.2	46.1	87.8	67.1	2346	69.4	-	73.9
CurveLanes-NAS-S [6]	88.3	68.6	63.2	68.0	47.9	82.5	66.0	2817	66.2	-	71.4
CurveLanes-NAS-M [6]	90.2	70.5	65.9	69.3	48.8	85.7	67.5	2359	68.2	-	73.5
CurveLanes-NAS-L [6]	90.7	72.3	67.7	70.1	49.4	85.8	68.4	1746	68.9	-	74.8
RESA (ResNet-34) [35]	91.9	72.4	66.5	72.0	46.3	88.1	68.6	1896	69.8	45	74.5
RESA (ResNet-50) [35]	92.1	73.1	69.2	72.8	47.7	88.3	**70.3**	1503	69.9	36	75.3
LaneATT (ResNet-18) [46]	91.1	72.9	65.7	70.9	48.3	85.4	63.3	**1170**	68.9	250	75.1
LaneATT (ResNet-34) [46]	92.1	75.0	66.4	78.1	49.3	88.3	67.7	1330	70.7	171	76.6
SGNet [47]	91.4	74.0	66.8	72.1	50.1	87.1	67.0	1164	70.6	117	76.1
FOLOLane [38]	**92.7**	**77.8**	**75.2**	**79.3**	**52.1**	**89.0**	69.4	1569	**74.5**	40	**78.8**
UFLD(ResNet-18) [20]	87.7	66.0	58.4	62.8	40.2	81.0	57.9	1743	62.1	323	68.4
UFLD(ResNet-34) [20]	90.7	70.2	59.5	69.3	44.4	85.7	69.5	2037	66.7	175	72.3
UFLDv2(ResNet-18)	91.7	73.0	64.6	74.7	47.2	87.6	68.7	1998	70.2	**330**	74.7
UFLDv2(ResNet-18)*	91.8	73.3	65.3	75.1	47.6	87.9	68.5	2075	70.7	310	75.0
UFLDv2(ResNet-34)	92.5	74.9	65.7	75.3	49.0	88.5	70.2	1864	70.6	165	75.9
UFLDv2(ResNet-34)*	92.5	74.8	65.5	75.5	49.2	88.8	70.1	1910	70.8	156	76.0

For crossroads, only false positives are shown. The less, the better. '-' means the result is not available.
** means our method with the feature level test time augmentation.*



(a) Column Anchor



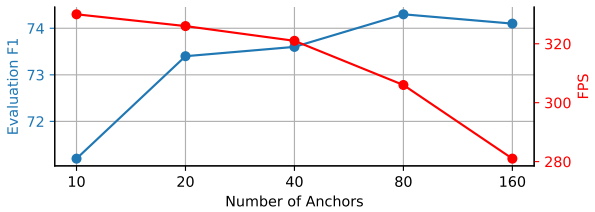
(b) Row Anchor

Fig. 11. Effects of different classification dimensions on row and column anchors.

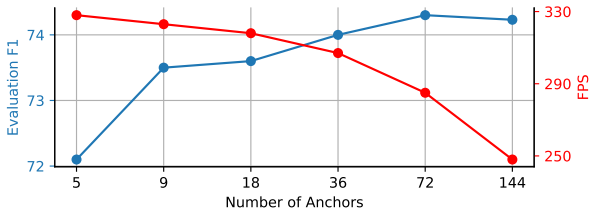
i.e. the localization ability of each class is worse. With a larger dimension, each class represents a narrower range of locations (better localization ability of each class), but the classification itself is harder. The final performance is a trade-off between the difficulty of classification and the localization ability of each class. So we set N^c_{dim} to 100 and N^r_{dim} to 200.

4.2.5 Effects of the Number of Anchors

There is another important hyperparameter in our method: the number of anchors (N_{row} and N_{col}) used to represent lanes. In this way, we set the number of row anchors N_{row} to 18 (following [2]) and conduct experiments with the number of column anchors N_{col} as 10, 20, 40, 80, and 160. The results are shown in Fig. 12a. Then, we set N_{col} to 40 and conduct experiments with N_{row} as 5, 9, 18, 36, and 72. The results are shown in Fig. 12b.



(a) Column Anchor



(b) Row Anchor

Fig. 12. Effects of using different number of row and column anchors.

We can see that with the increase of the number of row anchors, the performance also increases generally. But the detection speed also drops gradually. In this way, we set N_{row} and N_{col} to 18 and 40, respectively, to get a balance between performance and speed for ResNet-18-based models. For large models like ResNet-34, we set N_{row} and N_{col} to 72 and 80, respectively.

4.3 Results

This section shows the results on the four lane detection datasets, which are the TuSimple, CULane, CurveLanes, and LLAMAS datasetS. In these experiments, ResNet-18 and ResNet-34 are used as our backbone models.

TABLE 8
Comparison with other methods on TuSimple test set.

Method	F1	Accuracy	FP	FN	Runtime	Speed up
Res18-Seg [62]		92.69			25.3	5.3x
Res34-Seg [62]		92.84			50.5	2.6x
LaneNet [5]	94.80	96.38	7.80	2.44	19.0	7.0x
EL-GAN [63]	96.26	94.90	4.12	3.36	>100	<1.3x
SCNN [2]	95.97	96.53	6.17	**1.80**	133.5	1.0x
SAD [3]	95.92	**96.64**	6.02	2.05	13.4	10.0x
FastDraw [40]	93.92	95.20	7.60	5.40	11.1	12.0x
LSTR [44]	**96.85**	96.18	**2.91**	3.38	20.8	6.3x
SGNet [47]		95.87			10.9	12.3x
LaneATT (Res18) [46]	96.71	95.57	3.56	3.01	4.0	33.4x
LaneATT (Res34) [46]	96.77	95.63	3.53	2.92	5.9	22.6x
protocol 1						
UFLD(ResNet-18)	87.87	95.82	19.05	3.92	3.2	41.7x
UFLD(ResNet-34)	87.91	95.81	19.02	3.85	5.9	22.6x
UFLDv2(ResNet-18)	87.90	95.84	18.98	3.94	3.2	41.7x
UFLDv2(ResNet-34)	88.08	95.73	18.84	3.70	5.9	22.6x
protocol 2						
UFLD(ResNet-18)	96.05	95.50	3.06	4.82	3.2	41.7x
UFLD(ResNet-34)	96.13	95.53	3.06	4.66	5.9	22.6x
UFLDv2(ResNet-18)	96.16	95.65	3.06	4.61	3.2	41.7x
UFLDv2(ResNet-34)	96.22	95.56	3.18	4.37	5.9	22.6x

The calculation of speed up is based on the slowest method SCNN.
The speed of LSTR is measured on Nvidia RTX 3090 GPU with CUDA 11.1, PyTorch 1.8.2, and batchsize=1.

For the CULane dataset, seven methods, including SCNN [2], SAD [3], ERFNet [36], SIM-CycleGAN [61], CurveLanes [6], SGNet [47], RESA [35], LaneATT [46] and FOLOLane [38], are used for comparison. F1-measure and runtime are compared. The results can be seen in Table 7.

It is observed from Table 7 that our method achieves the fastest speed. Compared with the previous conference version, we can get stronger results with 6.3 points of performance improvement at the same speed. It proves the effectiveness of the proposed formulation on these challenging scenarios because our method could utilize global information to address the no-visual-clue and efficiency problems. The fastest model achieves 300+ FPS.

For the TuSimple lane detection benchmark, ten methods are used for comparison, including Res18-Seg [62], Res34-Seg [62], LaneNet [5], EL-GAN [63], SCNN [2], SAD [3], SGNet [47], LaneATT [46], and LSTR [44]. F1, Accuracy, FP, FN and runtime are compared in this experiment. The runtime of our method is recorded with the average time for 100 runs. To validate our methods, we use two protocols. Protocol 1 outputs all lanes and the absent lanse are represented with lanes full of `invalid`, which is the same as the conference version. Protocol 2 directly discards the absent lanes. The results are shown in Table 8.

From Table 8, we can see that our method achieves comparable performance with state-of-the-art methods while our method could run extremely fast. The biggest runtime gap between our method and SCNN is that our method could infer 41.7 times faster.

Another interesting phenomenon we should notice is that our method gains both better performance and faster speed when the backbone network is the same as plain segmentation, which is a DeeplabV2 [62] model with the resnet backbones. This result shows that our method is better than the segmentation formulation and verifies the effectiveness of our formulation.

TABLE 9
Comparison on the CurveLane dataset.

Method	F1	Presicion	Recall	FPS
SCNN [2]	65.02	76.13	56.74	8
SAD [3]	50.31	63.60	41.60	75
PointLaneNet [64]	78.47	86.33	72.91	111
CurveLane-S [6]	81.12	**93.58**	71.59	<45
UFLDv2(ResNet-18)	80.45	81.49	79.44	**145**
UFLDv2(ResNet-34)	**81.34**	81.93	**80.76**	86

For the CurveLanes dataset, we show the results in Table 9. We can see that our method achieves a better performance while maintaining a faster speed compared with the CurveLane-S method.

TABLE 10
Comparison on the LLAMAS dataset.

Method	F1	Precision	Recall	FPS
PolyLaneNet [43]	88.40	88.87	87.93	115
LaneATT(ResNet-18) [46]	93.46	**96.92**	90.24	250
LaneATT(ResNet-34) [46]	93.74	96.79	90.88	171
LaneATT(ResNet-122) [46]	93.54	96.82	90.47	26
UFLDv2(ResNet-18)	94.58	95.29	93.88	**285**
UFLDv2(ResNet-34)	**94.95**	95.75	**94.17**	156

For the LLAMAS dataset, we show the results in Table 10. It can be seen that our method also achieves the best performance and the fastest speed.

The visualizations of our method on the four datasets are shown in Figs. 13 and 14. We can see that the proposed hybrid anchors and our method perform well under various conditions.

Fig. 13. Visual comparisons between row and hybrid anchors under occluded, night, and normal settings. We show the results of the row-anchor model on the left and the hybrid-anchor model on the right. Yellow ellipses are used to highlight the differences.

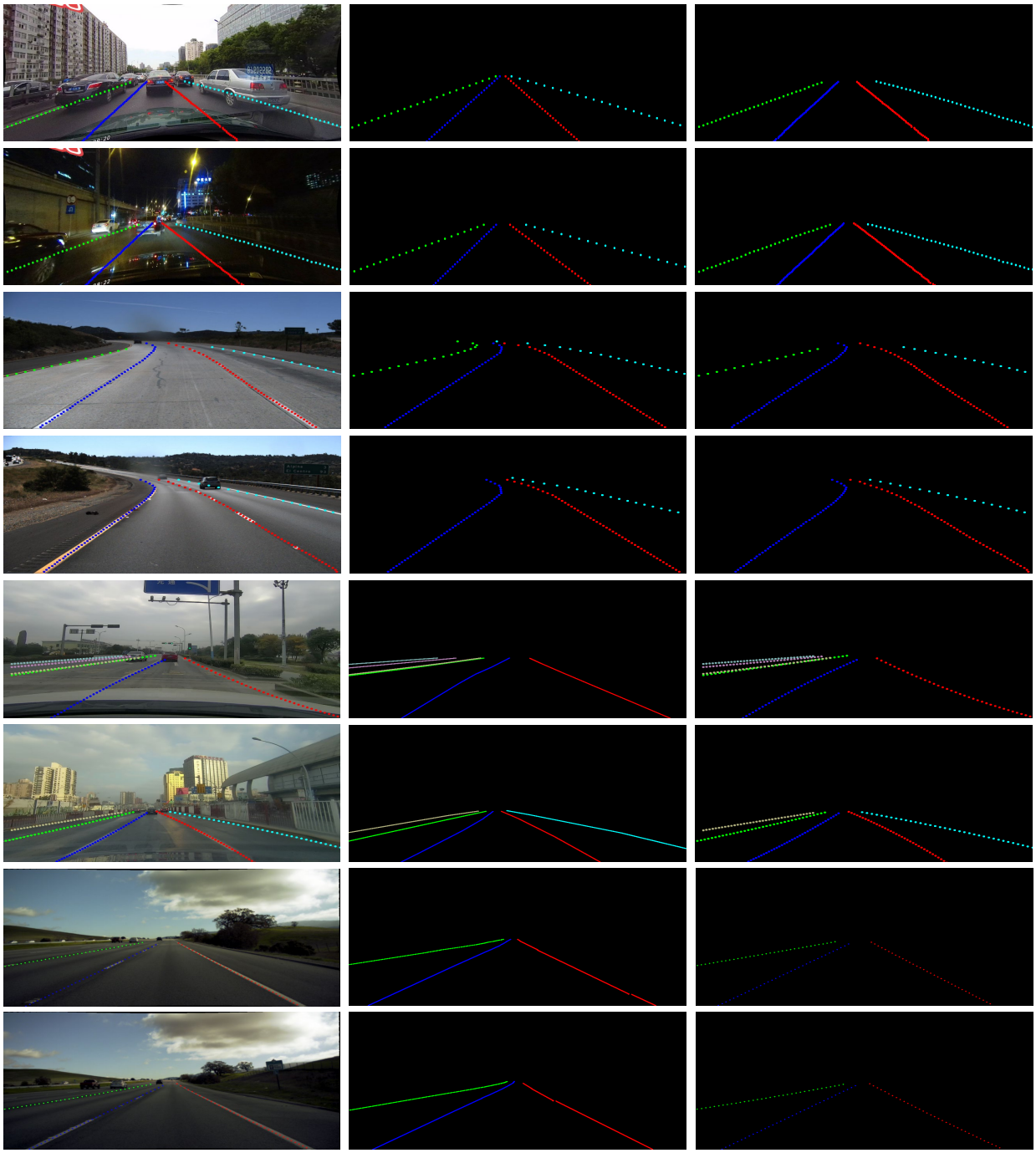


Fig. 14. Visualization on the CULane, TuSimple, CurveLanes, and LLAMAS datasets. Each dataset is visualized with two rows. From left to right, the results are images, ground truths, and predictions, respectively. In the image, predicted lane points are annotated.

5 Conclusion

In this paper, we have proposed a novel formulation with a hybrid anchor system and ordinal classification pipeline to achieve both remarkable speed and accuracy. The proposed formulation regards lane detection as directly learning the sparse coordinates on a hybrid anchor system with top-down classification based on the global feature. In this way, the problems of efficiency and no-visual-clue can be effectively addressed. The effectiveness of the proposed hybrid anchor system and ordinal classification losses are justified with both qualitative and quantitative experiments. Especially, a lightweight ResNet-18 version of our method could even achieve 300+ FPS.

There also exists some weakness of our method that the arrangements of anchors, i.e., how many anchors are to be set and where to place anchors, are still fixed and handcrafted. Automatic, dynamic, rotatable, and nonuniform anchors would be the potential directions for stronger and even faster methods.

Acknowledgments

This work was supported in part by National Key Research and Development Program of China under

Grant 2020AAA0107400, Zhejiang Provincial Natural Science Foundation of China under Grant LR19F020004, National Natural Science Foundation of China under Grant U20A20222, and OPPO Research Fund. The authors would like to thank Huanyu Wang for his contribution in idea discussion.

REFERENCES

[1] A. B. Hillel, R. Lerner, D. Levi, and G. Raz, "Recent progress in road and lane detection: a survey," *Mach. Vis. Appl.*, vol. 25, no. 3, pp. 727–745, 2014.

[2] X. Pan, J. Shi, P. Luo, X. Wang, and X. Tang, "Spatial as deep: Spatial cnn for traffic scene understanding," in *AAAI*, 2018, pp. 7276–7284.

[3] Y. Hou, Z. Ma, C. Liu, and C. C. Loy, "Learning lightweight lane detection cnns by self attention distillation," in *Int. Conf. Comput. Vis.*, 2019, pp. 1013–1021.

[4] S. Lee, J. Kim, J. Shin Yoon, S. Shin, O. Bailo, N. Kim, T.-H. Lee, H. Seok Hong, S.-H. Han, and I. So Kweon, "Vpgnet: Vanishing point guided network for lane and road marking detection and recognition," in *Int. Conf. Comput. Vis.*, Oct 2017, pp. 1947–1955.

[5] D. Neven, B. De Brabandere, S. Georgoulis, M. Proesmans, and L. Van Gool, "Towards end-to-end lane detection: an instance segmentation approach," in *IEEE Intell. Veh. Symp.*, 2018, pp. 286–291.

[6] H. Xu, S. Wang, X. Cai, W. Zhang, X. Liang, and Z. Li, "Curvelanenas: Unifying lane-sensitive architecture search and adaptive point blending," in *Eur. Conf. Comput. Vis.*, 2020, pp. 1–16.

[7] R. Girshick, "Fast r-cnn," in *Int. Conf. Comput. Vis.*, 2015, pp. 1440–1448.

[8] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, 2017.

[9] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *IEEE Conf. Comput. Vis. Pattern Recog.*, 2016, pp. 779–788.

[10] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "Ssd: Single shot multibox detector," in *Eur. Conf. Comput. Vis.* Springer, 2016, pp. 21–37.

[11] Z. Tian, C. Shen, H. Chen, and T. He, "Fcos: A simple and strong anchor-free object detector," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 1, no. 1, pp. 1–12, 2020.

[12] Y. Sun, X. Wang, and X. Tang, "Deep convolutional network cascade for facial point detection," in *IEEE Conf. Comput. Vis. Pattern Recog.*, 2013, pp. 3476–3483.

[13] Z. Zhang, P. Luo, C. C. Loy, and X. Tang, "Facial landmark detection by deep multi-task learning," in *Eur. Conf. Comput. Vis.* Springer, 2014, pp. 94–108.

[14] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint face detection and alignment using multitask cascaded convolutional networks," *IEEE Sign. Process. Letters*, vol. 23, no. 10, pp. 1499–1503, 2016.

[15] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, "Openpose: Realtime multi-person 2d pose estimation using part affinity fields," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 1, pp. 172–186, 2021.

[16] S.-E. Wei, V. Ramakrishna, T. Kanade, and Y. Sheikh, "Convolutional pose machines," in *IEEE Conf. Comput. Vis. Pattern Recog.*, 2016, pp. 4724–4732.

[17] A. Newell, K. Yang, and J. Deng, "Stacked hourglass networks for human pose estimation," in *Eur. Conf. Comput. Vis.* Springer, 2016, pp. 483–499.

[18] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *IEEE Conf. Comput. Vis. Pattern Recog.* Ieee, 2009, pp. 248–255.

[19] R. Rothe, R. Timofte, and L. Van Gool, "Dex: Deep expectation of apparent age from a single image," in *IEEE Conf. Comput. Vis. Worksh.*, 2015, pp. 252–257.

[20] Z. Qin, H. Wang, and X. Li, "Ultra fast structure-aware deep lane detection," in *Eur. Conf. Comput. Vis.*, 2020, pp. 1–16.

[21] T.-Y. Sun, S.-J. Tsai, and V. Chan, "Hsi color model based lane-marking detection," in *IEEE Trans. Intell. Transp. Syst. Conf.*, 2006, pp. 1168–1172.

[22] B. Yu and A. K. Jain, "Lane boundary detection using a multiresolution hough transform," in *IEEE Int. Conf. Image Process.*, vol. 2, 1997, pp. 748–751.

[23] Y. Wang, D. Shen, and E. K. Teoh, "Lane detection using spline model," *Pattern Recognit. Lett.*, vol. 21, no. 8, pp. 677–689, 2000.

[24] M. Bertozzi and A. Broggi, "Gold: A parallel real-time stereo vision system for generic obstacle and lane detection," *IEEE Trans. Image Process.*, vol. 7, no. 1, pp. 62–81, 1998.

[25] M. Aly, "Real time detection of lane markers in urban streets," in *IEEE Intell. Veh. Symp.*, 2008, pp. 7–12.

[26] Y. Wang, E. K. Teoh, and D. Shen, "Lane detection and tracking using b-snake," *Image Vis. Comput.*, vol. 22, no. 4, pp. 269–280, 2004.

[27] Z. Kim, "Robust lane detection and tracking in challenging scenarios," *IEEE Trans. Intell. Transp. Syst.*, vol. 9, no. 1, pp. 16–26, 2008.

[28] P. Krähenbühl and V. Koltun, "Efficient inference in fully connected crfs with gaussian edge potentials," in *Adv. Neural Inform. Process. Syst.*, 2011, pp. 109–117.

[29] K. Kluge and S. Lakshmanan, "A deformable-template approach to lane detection," in *IEEE Intell. Veh. Symp.*, 1995, pp. 54–59.

[30] J. P. Gonzalez and U. Ozguner, "Lane detection using histogram-based segmentation and decision trees," in *IEEE Trans. Intell. Transp. Syst. Conf.*, 2000, pp. 346–351.

[31] H. M. Mandalia and M. D. D. Salvucci, "Using support vector machines for lane-change detection," in *Proc. Hum. Factors Ergon. Soc. Annu. Meet.*, vol. 49, no. 22, 2005, pp. 1965–1969.

[32] J. Kim and M. Lee, "Robust lane detection based on convolutional neural network and random sample consensus," in *ICONIP*, 2014, pp. 454–461.

[33] B. Huval, T. Wang, S. Tandon, J. Kiske, W. Song, J. Pazhayampallil, M. Andriluka, P. Rajpurkar, T. Migimatsu, R. Cheng-Yue *et al.*, "An empirical evaluation of deep learning on highway driving," *arXiv preprint arXiv:1504.01716*, 2015.

[34] Z. M. Chng, J. M. H. Lew, and J. A. Lee, "Roneld: Robust neural network output enhancement for active lane detection," 2020.

[35] T. Zheng, H. Fang, Y. Zhang, W. Tang, Z. Yang, H. Liu, and D. Cai, "Resa: Recurrent feature-shift aggregator for lane detection," 2020.

[36] Y. Hou, Z. Ma, C. Liu, T.-W. Hui, and C. C. Loy, "Inter-region affinity distillation for road marking segmentation," in *IEEE Conf. Comput. Vis. Pattern Recog.*, June 2020.

[37] H. Abualsaud, S. Liu, D. Lu, K. Situ, A. Rangesh, and M. M. Trivedi, "Laneaf: Robust multi-lane detection with affinity fields," *arXiv preprint arXiv:2103.12040*, 2021.

[38] Z. Qu, H. Jin, Y. Zhou, Z. Yang, and W. Zhang, "Focus on local: Detecting lane marker from bottom up via key point," *arXiv preprint arXiv:2105.13680*, 2021.

[39] J. Li, X. Mei, D. Prokhorov, and D. Tao, "Deep neural network for structural prediction and lane detection in traffic scene," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 3, pp. 690–703, 2016.

[40] J. Philion, "Fastdraw: Addressing the long tail of lane detection by adapting a sequential prediction network," in *IEEE Conf. Comput. Vis. Pattern Recog.*, 2019, pp. 11 582–11 591.

[41] Y.-C. Hsu, Z. Xu, Z. Kira, and J. Huang, "Learning to cluster for proposal-free instance segmentation," in *IJCNN*, 2018, pp. 1–8.

[42] W. Van Gansbeke, B. De Brabandere, D. Neven, M. Proesmans, and L. Van Gool, "End-to-end lane detection through differentiable least-squares fitting," *arXiv preprint arXiv:1902.00293*, 2019.

[43] L. Tabelini, R. Berriel, T. M. Paixao, C. Badue, A. F. De Souza, and T. Oliveira-Santos, "Polylanenet: Lane estimation via deep polynomial regression," in *Int. Conf. Pattern Recog.* IEEE, 2021, pp. 6150–6156.

[44] R. Liu, Z. Yuan, T. Liu, and Z. Xiong, "End-to-end lane shape prediction with transformers," in *WACV*, 2021, pp. 3694–3702.

[45] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Adv. Neural Inform. Process. Syst.*, 2017, pp. 5998–6008.

[46] L. Tabelini, R. Berriel, T. M. P. ao, C. Badue, A. F. D. Souza, and T. Oliveira-Santos, "Keep your eyes on the lane: Real-time attention-guided lane detection," in *IEEE Conf. Comput. Vis. Pattern Recog.*, 2021, pp. 1–9.

[47] J. Su, C. Chen, K. Zhang, J. Luo, X. Wei, and X. Wei, "Structure guided lane detection," *arXiv preprint arXiv:2105.05403*, 2021.

[48] N. Garnett, R. Cohen, T. Pe'er, R. Lahav, and D. Levi, "3d-lanenet: end-to-end 3d multiple lane detection," in *Int. Conf. Comput. Vis.*, 2019, pp. 2921–2930.

[49] Y. Guo, G. Chen, P. Zhao, W. Zhang, J. Miao, J. Wang, and T. E. Choe, "Gen-lanenet: A generalized and scalable approach for 3d lane detection," pp. 1–16, 2020.

[50] N. Efrat, M. Bluvstein, N. Garnett, D. Levi, S. Oron, and B. E. Shlomo, "Semi-local 3d lane detection and uncertainty estimation," *arXiv preprint arXiv:2003.05257*, 2020.
[51] N. E. Sela, M. Bluvstein, S. Oron, D. Levi, N. Garnett, and B. E. Shlomo, "3d-lanenet+: Anchor free lane detection using a semi-local representation," *arXiv preprint arXiv:2011.01535*, 2020.
[52] L. Liu, X. Chen, S. Zhu, and P. Tan, "Condlanenet: a top-to-down lane detection framework based on conditional convolution," *arXiv preprint arXiv:2105.05003*, 2021.
[53] TuSimple, "Tusimple benchmark," https://github.com/TuSimple/tusimple-benchmark, accessed November, 2019.
[54] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Int. Conf. Learn. Represent.*, 2015, pp. 1–14.
[55] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *IEEE Conf. Comput. Vis. Pattern Recog.*, 2015, pp. 1–9.
[56] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conf. Comput. Vis. Pattern Recog.*, 2016, pp. 770–778.
[57] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He, "Aggregated residual transformations for deep neural networks," in *IEEE Conf. Comput. Vis. Pattern Recog.*, 2017, pp. 1492–1500.
[58] E. Romera, J. M. Alvarez, L. M. Bergasa, and R. Arroyo, "Erfnet: Efficient residual factorized convnet for real-time semantic segmentation," *IEEE Trans. Intell. Transp. Syst.*, vol. 19, no. 1, pp. 263–272, 2017.
[59] K. Behrendt and R. Soussan, "Unsupervised labeled lane markers using maps," in *IEEE Conf. Comput. Vis. Worksh.*, 2019.
[60] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer, "Automatic differentiation in pytorch," in *Adv. Neural Inform. Process. Syst. Worksh.*, 2017, pp. 1–4.
[61] T. Liu, Z. Chen, Y. Yang, Z. Wu, and H. Li, "Lane detection in low-light conditions using an efficient data enhancement: Light conditions style transfer," in *IEEE Intell. Veh. Symp.*, 2020, pp. 1394–1399.
[62] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, "Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 4, pp. 834–848, 2017.
[63] M. Ghafoorian, C. Nugteren, N. Baka, O. Booij, and M. Hofmann, "El-gan: embedding loss driven generative adversarial networks for lane detection," in *Eur. Conf. Comput. Vis. Worksh.*, 2018, pp. 1–17.
[64] Z. Chen, Q. Liu, and C. Lian, "Pointlanenet: Efficient end-to-end cnns for accurate real-time lane detection," in *IEEE Intell. Veh. Symp.* IEEE, 2019, pp. 2563–2568.

Zequn Qin received his master's degree in 2019 from Northwestern Polytechnical University, China. He is currently a Ph.D. candidate at Zhejiang University. His research interests include computer vision and autonomous driving.

Pengyi Zhang received the bachelor's degree in computer science and technology from Beijing Institute of Technology, China, in 2020. He is currently pursuing the master's degree with the College of Computer Science, Zhejiang University, Hangzhou, China, under the supervision of Prof. X. Li. His current research interests are primarily in computer vision and deep learning.

Xi Li received the Ph.D. degree from the National Laboratory of Pattern Recognition, Chinese Academy of Sciences, Beijing, China, in 2009. From 2009 to 2010, he was a Post-Doctoral Researcher with CNRS, Telecomd ParisTech, France. He is currently a Full Professor with Zhejiang University, China. Prior to that, he was a Senior Researcher with the University of Adelaide, Australia. His research interests include artificial intelligence, computer vision, and machine learning.

Ultra Fast Deep Lane Detection with Hybrid Anchor Driven Ordinal Classification –Supplementary Materials

1 Overview

These supplementary materials provide more statistics, discussions, illustrations, and tables to better cover and elaborate the motivations, advantages, and details about our work. The added topics are: statistics about the derived error band, measured FPS on CPUs and embedded chips, illustrations of the advantages of the hybrid anchor, and details about the post-processing in this work.

2 Validating the magnified localization problem

To validate the magnified localization problem and the error of $\varepsilon/\sin\theta$, we measure the localization errors corresponding to different angles of lanes, and the statistics are shown in fig. A. We can see that the trends of statistical results fit the mathematically derived error band of $\varepsilon/\sin\theta$. This shows the magnified localization problem indeed exists.

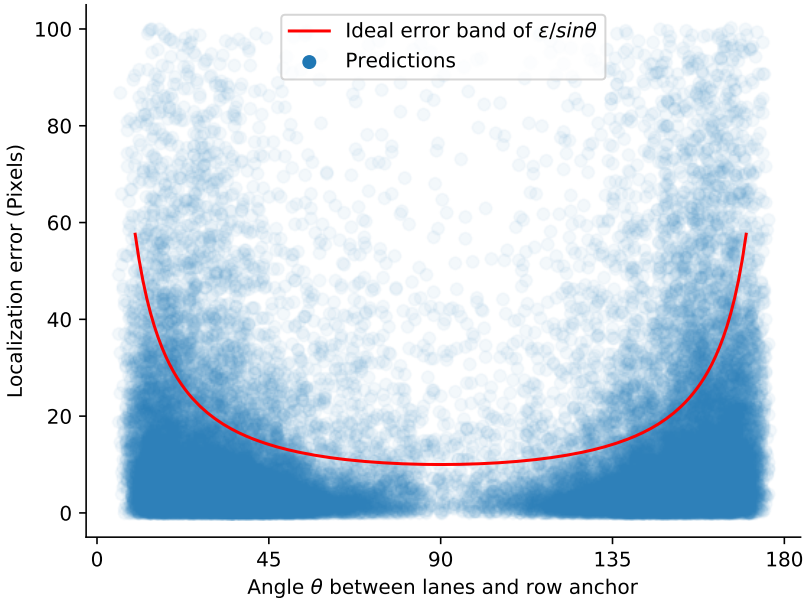


Fig. A. Illustration of the statistics of the errors vs. angles in a model with row anchors.

3 The speed on the CPUs and embedd chips

We test our method on various CPUs and embedded chips like Nvidia TX2, as shown in tables A and B. We can see that our method could achieve real-time or near-real-time speed even in CPUs and embedded chips.

TABLE A
FPS on different devices at 320x1600 resolution with float32 precision.

Device	AMD EPYC 7282	Intel Xeon E5-2678	Intel i9 10990K	NVIDIA Jetson TX2
FPS	7.7	7.9	15.0	36.3

TABLE B
FPS on different devices at 320x800 resolution with float32 precision.

Device	AMD EPYC 7282	Intel Xeon E5-2678	Intel i9 10990K	NVIDIA Jetson TX2
FPS	15.4	15.9	29.8	36.3

4 Demonstrating of the advantage of hybrid anchor

To better illustrate the properties of different types of anchors, we show an example in fig. B. From fig. B, we can see that different anchor systems have different preferences in modeling lines. Specifically, row anchor is more appropriate for vertical lanes since vertical lane have large variance in vertical directions and can be more intersected by row anchors than column anchors. And this is also true for column anchors with horizontal lanes. By appropriately adopting different anchor systems for different kinds of lines, hybrid anchors could be more efficient and effective than single row or column anchors.

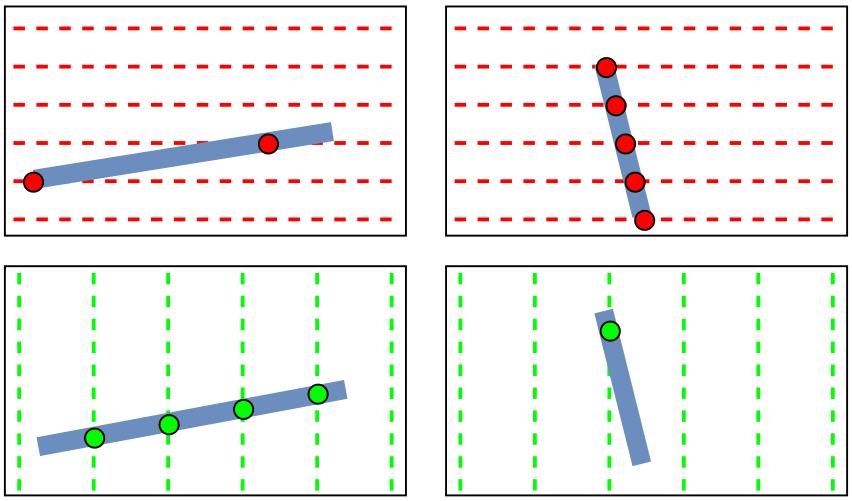


Fig. B. Illustration of the properties of different types of anchors. Red and green dashed lines represent the row and column anchors, respectively. With the same number of six anchors, row anchors could model vertical lines better while column anchors could fit horizontal lines better.

5 The hidden premise of the anchor-based method

There is a hidden premise of our method, i.e., each horizontal / vertical line should only have one intersection point with one lane instance. In this way, we discuss the effectiveness of our method when facing multiple-intersections for extremely curved or complex roads.

Generally, there are two kinds of results when facing the multiple-intersection problem, which are top- and bottom-first representations. The top-first method preserves the top intersection while the bottom-first one preserves the bottom intersection. To better illustrate the problem of multiple-intersection, we show an example in fig. C.

From fig. C we can see that for both top- and bottom-first methods, although the restored lanes with the multiple-intersection problem are slightly inaccurate or incomplete, they are still valid. For the top-first method, the results are not much affected. The bottom-first method's results are slightly shorter than the ground truth, which is acceptable since the major lower parts of lanes are still intact. In practice, we use bottom-first representation in our work.

6 Post-processing

In post-processing, we would remove invalid lane predictions that are very short. For the original row-based method, we set a very loose threshold (at least two lane points) since a horizontal lane

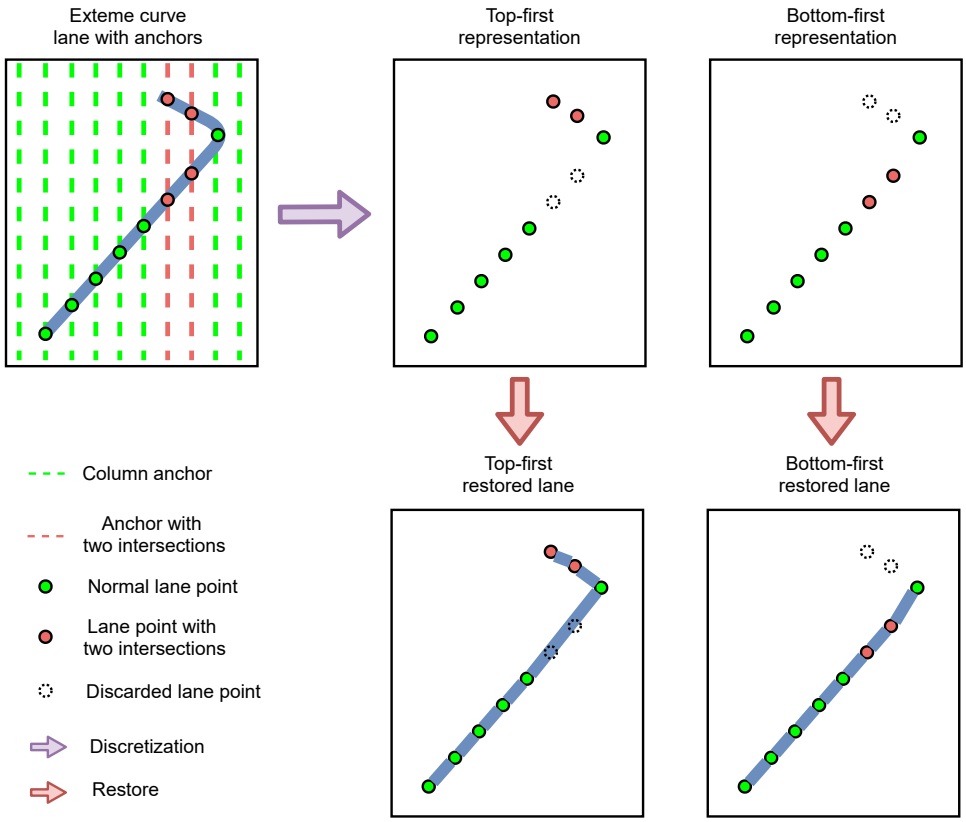


Fig. C. illustration of the multiple-intersection problem.

could have very few intersections with the row anchors. While for the hybrid anchor method, we can set a more aggressive threshold because hybrid anchor could represent both horizontal and vertical lanes well with denser lane points.

This difference causes the different results of our conference version and the performance in Table 4 of the journal paper (68.4 vs. 66.09). In short, 68.4 is the row-based method with the original loose post-processing, and 66.09 is the row-based method with the aggressive post-processing of hybrid anchor.